

Database Systems — Lab Work Solutions

Week 2: Relational Model & Keys — Full Task Solutions

Student: Amr Khaled

Part 1 — Key Identification

Task 1.1 — Relation A: Employee

Employee(EmpID, SSN, Email, Phone, Name, Department, Salary)

1. Six different superkeys

- {EmpID}
- {SSN}
- {Email}
- {EmpID, Name}
- {SSN, Department}
- {EmpID, SSN, Email, Name, Department, Salary, Phone} (the entire set of attributes is always a superkey)

Any superset of a key that still uniquely identifies every row is a valid superkey — adding extra columns never breaks uniqueness, it just makes the key non-minimal.

2. All candidate keys

A candidate key is a superkey with no unnecessary attributes (minimal). Testing each single attribute against the sample data:

- {EmpID} — unique per row, minimal ✓
- {SSN} — unique per row, minimal ✓
- {Email} — unique per row, minimal ✓

Name, Department, Salary, and Phone are NOT candidate keys on their own — Name and Phone could realistically repeat across employees, and Department/Salary are shared by design.

3. Choice of Primary Key

Chosen Primary Key: EmpID.

Reasoning: EmpID is (a) system-generated and stable — it never changes even if the employee's name, email, or department changes; (b) simple — a short numeric value is cheaper to index and join on than a text string; (c) not sensitive — unlike SSN, exposing EmpID in logs, URLs, or foreign keys carries no privacy risk. SSN would also work technically, but it is personally identifiable information and many organizations avoid using it as a database key for privacy/legal reasons. Email is a poor choice because employees can change their email address, which would force every referencing foreign key to be updated as well.

4. Can two employees share the same phone number?

Yes — based on general business logic, two employees CAN share the same phone number (e.g., a shared department landline or reception extension), even though in this particular 3-row sample every phone happens to be different. The sample data alone can never prove a uniqueness constraint; uniqueness must come from a business rule. Since no such rule is stated for Phone, and shared office lines are common in practice, Phone should not be treated as a candidate key.

Task 1.1 — Relation B: Course Registration

Registration(StudentID, CourseCode, Section, Semester, Year, Grade, Credits)

Business rules: (1) a student can retake the same course in a different semester; (2) a student cannot register for the same section of a course in the same semester; (3) each section's credit value is fixed per term.

1. Minimum attributes needed for the Primary Key

PK = {StudentID, CourseCode, Section, Semester, Year}

2. Why each attribute is necessary

- StudentID — identifies which student the registration record belongs to; without it, two different students taking the same section would collide.
- CourseCode — identifies which course is being registered for.
- Section — a course can be offered in multiple sections (different instructor/time) during the same term; needed to distinguish which specific section the student is in.
- Semester and Year together — needed because rule (1) explicitly allows the SAME student to take the SAME course again in a later term. Without Semester+Year, a student's second attempt at a course would violate the key (duplicate row), which contradicts the stated business rule.

3. Additional candidate keys?

No other candidate key exists. Grade and Credits are outcome/descriptive attributes (they depend on the registration, they don't identify it), so no combination of attributes other than the one above can serve as a minimal unique identifier.

Task 1.2 — Foreign Key Design (University System)

Given tables: *Student(StudentID, Name, Email, Major, AdvisorID)* · *Professor(ProfID, Name, Department, Salary)* · *Course(CourseID, Title, Credits, DepartmentCode)* · *Department(DeptCode, DeptName, Budget, ChairID)* · *Enrollment(StudentID, CourseID, Semester, Grade)*

Foreign Key (in table)	References (Primary Key in)	Business meaning
Student.AdvisorID	Professor.ProfID	Each student is assigned one academic advisor, who is a professor.
Professor.Department	Department.DeptCode	Each professor belongs to exactly one department.
Course.DepartmentCode	Department.DeptCode	Each course is offered by exactly one department.
Department.ChairID	Professor.ProfID	Each department's chair must be one of its professors.
Enrollment.StudentID	Student.StudentID	Each enrollment record belongs to one student.
Enrollment.CourseID	Course.CourseID	Each enrollment record refers to one course.

Note: Department.ChairID → Professor.ProfID and Professor.Department → Department.DeptCode create a circular reference between the two tables. In practice this is resolved by allowing one of the two foreign keys to be NULL initially (e.g., create the department first with a NULL chair, then add professors, then update the chair) or by deferring the constraint check until the end of the transaction.

Part 2 — ER Diagram Construction

Task 2.1 — Hospital Management System

1. Entities (Strong vs. Weak)

Entity	Type	Reason
--------	------	--------

Patient	Strong	Has its own unique PatientID.
Doctor	Strong	Has its own unique DoctorID.
Department	Strong	Has its own unique DeptCode.
Appointment	Strong	Has its own unique AppointmentID (acts as an associative entity linking Patient and Doctor).
Prescription	Weak	Has no independent identifier on its own — it only makes sense in the context of a specific Appointment (owner entity).
HospitalRoom	Weak	Room number “101” is not unique hospital-wide (101 in Cardiology ≠ 101 in Neurology); it needs DeptCode + RoomNumber together to be unique.

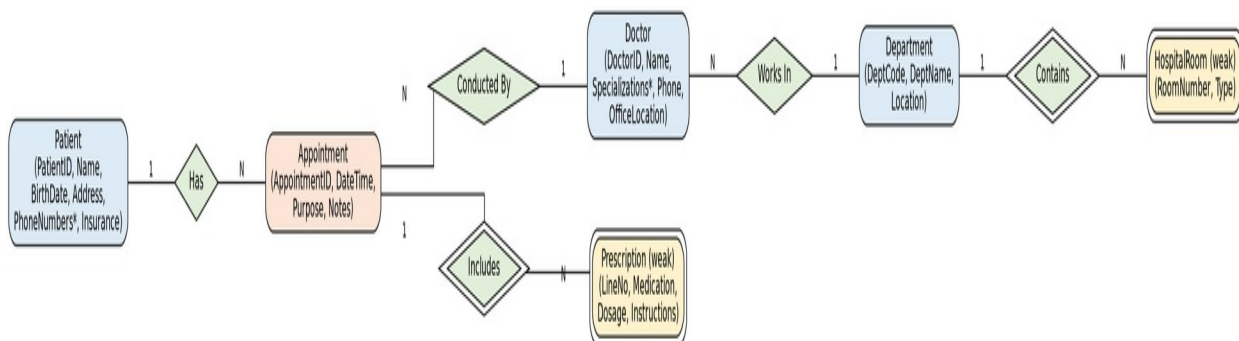
2. Attributes (classified)

Entity	Simple	Composite	Multi-valued	Derived
Patient	PatientID, BirthDate	Address (Street, City, State, Zip), Insurance	PhoneNumbers	Age (from BirthDate)
Doctor	DoctorID, OfficeLocation	Name (First, Last)	Specializations, Phone	—
Department	DeptCode, DeptName, Location	—	—	—
Appointment	AppointmentID, Purpose, Notes	DateTime (Date, Time)	—	—
Prescription	Medication, Dosage, Instructions	—	—	—
HospitalRoom	RoomNumber, Type	—	—	—

3. Relationships and Cardinalities

Relationship	Between	Cardinality
Has	Patient – Appointment	1 : N (one patient can have many appointments)
Conducted By	Appointment – Doctor	N : 1 (one doctor conducts many appointments)
Works In	Doctor – Department	N : 1 (many doctors belong to one department)
Includes	Appointment – Prescription	1 : N, identifying (a prescription only exists as part of one appointment)
Contains	Department – HospitalRoom	1 : N, identifying (a room belongs to exactly one department)

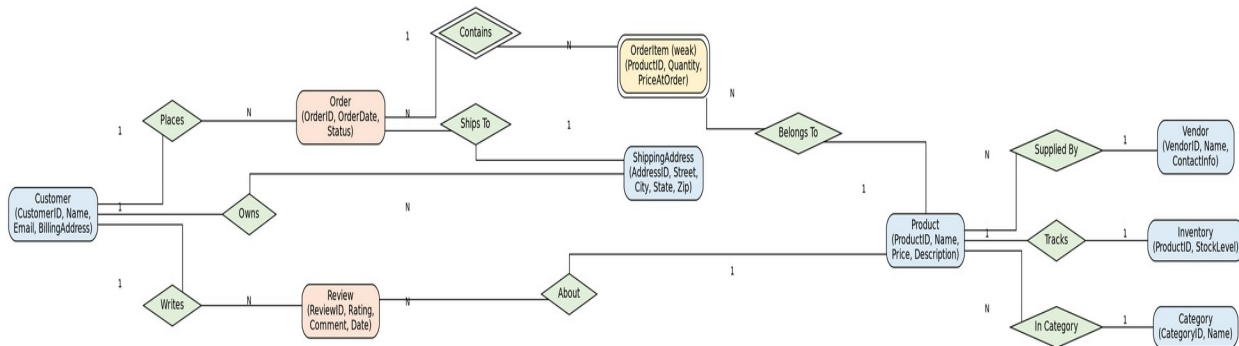
Together, Patient – Has – Appointment – Conducted By – Doctor models the real M:N relationship between Patient and Doctor (a patient sees many doctors, a doctor sees many patients), with Appointment as the associative entity that also carries its own attributes (DateTime, Purpose, Notes).



4 & 5. Primary keys

Patient(PatientID) · Doctor(DoctorID) · Department(DeptCode) · Appointment(AppointmentID) · Prescription(AppointmentID, LineNo) — composite, inherits owner's key · HospitalRoom(DeptCode, RoomNumber) — composite, inherits owner's key.

Task 2.2 — E-commerce Platform



Weak entity and justification

OrderItem is the weak entity. A single order item (“2 units of Product X at \$19.99”) has no meaning or identity outside the Order it belongs to — it cannot be looked up, referenced, or make sense on its own. Its key is composite: (OrderID, ProductID), combining its own partial key (ProductID) with the owner's key (OrderID).

Many-to-many relationship needing its own attributes

Order ↔ Product is fundamentally M:N (one order contains many products, one product appears in many orders). This relationship cannot be a simple line in the diagram because it carries its own data — Quantity and PriceAtOrder (the price may differ from the product's current listed price, since prices change over time but a past order must keep the price the customer actually paid). This is exactly why the M:N relationship is promoted into its own entity, OrderItem, rather than left as a plain relationship line.

Part 4 — Normalization Workshop

Task 4.1 — StudentProject (Denormalized Table)

StudentProject(StudentID, StudentName, StudentMajor, ProjectID, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept, Role, HoursWorked, StartDate, EndDate)

1. Functional Dependencies

- StudentID → StudentName, StudentMajor
- ProjectID → ProjectTitle, ProjectType, SupervisorID
- SupervisorID → SupervisorName, SupervisorDept
- (StudentID, ProjectID) → Role, HoursWorked, StartDate, EndDate

2. Problems in the current design

Redundancy: StudentName/StudentMajor repeat once for every project the same student works on; ProjectTitle/ProjectType/SupervisorID/SupervisorName/SupervisorDept repeat once for every student assigned to the same project.

Update anomaly: If a supervisor moves to a new department, SupervisorDept must be updated in EVERY row that mentions that supervisor. Missing even one row leaves the data inconsistent (the same supervisor appearing in two different departments).

Insert anomaly: A new project cannot be recorded until at least one student has been assigned to it, because StudentID is part of the primary key — you cannot insert a row with a NULL StudentID just to register the project's title/type/supervisor.

Delete anomaly: If the only student on a given project is removed (e.g., they drop the project), the entire row disappears — and with it, all information about that project (title, type, supervisor) is lost, even though the project itself may still exist.

3. 1NF check

Every attribute as given holds a single, atomic value (no repeating groups or lists inside one cell), so the table already satisfies 1NF. No fix is required at this stage.

4. 2NF

Primary Key: (StudentID, ProjectID) — a row represents one student's participation in one project.

Partial dependencies: StudentID → StudentName, StudentMajor (depends on only part of the key) and ProjectID → ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept (also depends on only part of the key). Role, HoursWorked, StartDate, and EndDate depend on the FULL key, so they are fine.

2NF decomposition:

- Student(StudentID, StudentName, StudentMajor) — PK: StudentID
- Project(ProjectID, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept) — PK: ProjectID
- StudentProject(StudentID, ProjectID, Role, HoursWorked, StartDate, EndDate) — PK: (StudentID, ProjectID); FK to Student and Project

5. 3NF

Transitive dependency: Inside the new Project table, ProjectID → SupervisorID → SupervisorName, SupervisorDept. SupervisorName/SupervisorDept depend on SupervisorID, not directly on ProjectID (the primary key) — this is a transitive dependency.

Final 3NF decomposition (4 tables):

- Student(StudentID, StudentName, StudentMajor) — PK: StudentID
- Supervisor(SupervisorID, SupervisorName, SupervisorDept) — PK: SupervisorID
- Project(ProjectID, ProjectTitle, ProjectType, SupervisorID) — PK: ProjectID; FK SupervisorID → Supervisor
- StudentProject(StudentID, ProjectID, Role, HoursWorked, StartDate, EndDate) — PK: (StudentID, ProjectID); FK StudentID → Student, FK ProjectID → Project

Every non-key attribute in every table now depends on the whole key, nothing but the key, and nothing transitively — all four tables are in 3NF.

Task 4.2 — CourseSchedule (Advanced Normalization)

CourseSchedule(StudentID, StudentMajor, CourseID, CourseName, InstructorID, InstructorName, TimeSlot, Room, Building)

1. Primary Key (the “tricky” part)

At first glance it looks like the key must include TimeSlot, Room, and/or InstructorID because those describe the 'section'. But re-reading the business rules: a student cannot be enrolled in the same course twice (each row is one student in one course), and each course offers each of its sections at a distinct time. So a single student can only ever be linked to ONE specific (TimeSlot, Room, Instructor) combination for a given CourseID. That means:

Primary Key = (StudentID, CourseID) — this pair alone is already enough to make every row unique, and everything else (TimeSlot, Room, InstructorID, Building) is determined once you know which student takes which course.

2. Functional Dependencies

- $\text{StudentID} \rightarrow \text{StudentMajor}$
- $\text{CourseID} \rightarrow \text{CourseName}$
- $\text{InstructorID} \rightarrow \text{InstructorName}$
- $\text{Room} \rightarrow \text{Building}$ (rooms are unique campus-wide, so a room's building never changes)
- $(\text{StudentID}, \text{CourseID}) \rightarrow \text{TimeSlot}, \text{Room}, \text{InstructorID}, \text{Building}$ (the full key determines the section details for that student)

3. Is the table in BCNF?

BCNF requires that for every FD $(X \rightarrow Y)$, X must be a superkey of the WHOLE table. Checking each FD above against the key $(\text{StudentID}, \text{CourseID})$:

Functional Dependency	Is the left side a superkey?	BCNF?
$\text{StudentID} \rightarrow \text{StudentMajor}$	No (StudentID alone doesn't determine CourseID)	Violation
$\text{CourseID} \rightarrow \text{CourseName}$	No (CourseID alone doesn't determine StudentID)	Violation
$\text{InstructorID} \rightarrow \text{InstructorName}$	No	Violation
$\text{Room} \rightarrow \text{Building}$	No	Violation

The table is NOT in BCNF — it has multiple violations, each caused by an attribute that determines other attributes without being a key itself.

4. BCNF Decomposition

- $\text{Student}(\text{StudentID}, \text{StudentMajor})$ — PK: StudentID
- $\text{Course}(\text{CourseID}, \text{CourseName})$ — PK: CourseID
- $\text{Instructor}(\text{InstructorID}, \text{InstructorName})$ — PK: InstructorID
- $\text{RoomBuilding}(\text{Room}, \text{Building})$ — PK: Room
- $\text{Section}(\text{CourseID}, \text{TimeSlot}, \text{Room}, \text{InstructorID})$ — PK: $(\text{CourseID}, \text{TimeSlot})$; also a candidate key: $(\text{Room}, \text{TimeSlot})$ and $(\text{InstructorID}, \text{TimeSlot})$, since a room or instructor can only host one class at a given time — every determinant here IS a superkey, so this table satisfies BCNF
- $\text{Enrollment}(\text{StudentID}, \text{CourseID}, \text{TimeSlot})$ — PK: $(\text{StudentID}, \text{CourseID})$; FK $(\text{CourseID}, \text{TimeSlot}) \rightarrow \text{Section}$

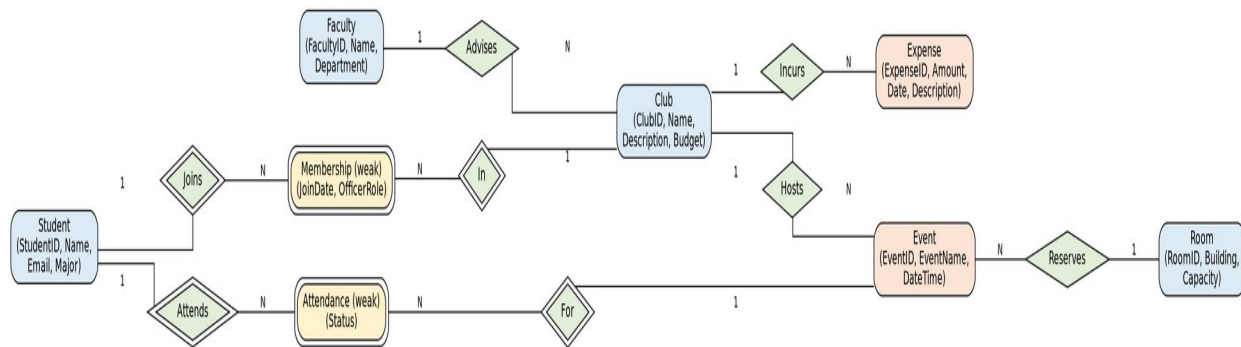
(Assumption made explicit: a given course does not run two different sections at the exact same TimeSlot. This is the standard simplifying assumption for this classic scheduling example — if it did not hold, a surrogate SectionID would be introduced instead of relying on TimeSlot alone.)

5. Potential loss of information

None — this decomposition follows the standard BCNF-violation-splitting algorithm (for each violating FD $X \rightarrow Y$, split into $R_1(X, Y)$ and $R_2(X, \text{rest})$), which is mathematically guaranteed to be a lossless-join decomposition: joining Student , Course , Instructor , RoomBuilding , Section , and Enrollment back together on their foreign keys exactly reconstructs the original CourseSchedule table with no spurious or missing rows.

Part 5 — Design Challenge

Task 5.1 — Student Clubs and Organizations



2. Normalized relational schema

Table	Attributes	Key
Student	StudentID, Name, Email, Major	PK: StudentID
Faculty	FacultyID, Name, Department	PK: FacultyID
Club	ClubID, Name, Description, Budget, AdvisorID	PK: ClubID; FK AdvisorID → Faculty
Membership	StudentID, ClubID, JoinDate, OfficerRole	PK: (StudentID, ClubID); FK's to Student, Club
Room	RoomID, Building, Capacity	PK: RoomID
Event	EventID, ClubID, EventName, DateTime, RoomID	PK: EventID; FK's to Club, Room
Attendance	EventID, StudentID, Status	PK: (EventID, StudentID); FK's to Event, Student
Expense	ExpenseID, ClubID, Amount, Date, Description	PK: ExpenseID; FK ClubID → Club

3. A design decision with multiple valid options

Decision: whether to store a club officer's position (President, Treasurer, Secretary, ...) as a plain attribute (OfficerRole) inside the Membership table, or as a completely separate OfficerPosition entity/table.

Choice made: store it as a single OfficerRole attribute inside Membership.

Justification: each student holds at most one officer role in a given club at any one time, so a single column is sufficient and keeps queries simple (no extra join needed just to find a club's president). A separate OfficerPosition table would only be justified if a club needed multiple people sharing one title simultaneously, or if the system had to track a full historical record of who held which position and when (e.g., "Jane was Treasurer from Sept 2024 to May 2025, then President from Sept 2025") — since the requirements here don't mention historical tracking, the simpler design was chosen.

4. Three example queries (in English, not SQL)

- Query 1: Find all students who are officers (President, Treasurer, or Secretary) in the Computer Science Club.
- Query 2: List all events scheduled for next week along with their reserved rooms and the clubs hosting them.
- Query 3: Show the total amount spent (sum of all expenses) by each club so far this semester.